

ZSL

Zentrum für Schulqualität
und Lehrerbildung
Baden-Württemberg


cisco

Networking
Academy



ACL Concepts

Andreas Grupp

Andreas.Grupp@zsl-tue.de

Carina Haag

haag.c@lanz.schule

Tobias Heine

tobias.heine@springer-schule.de

Uwe Thiessat

uwe.thiessat@gbs-sha.de

- Wie können ACLs Netzwerkverkehr filtern?
- Wie werden bei ACLs Wildcard Masken eingesetzt?
- Wie werden ACLs erstellt?
- Vergleich von standard- und extended ACLs

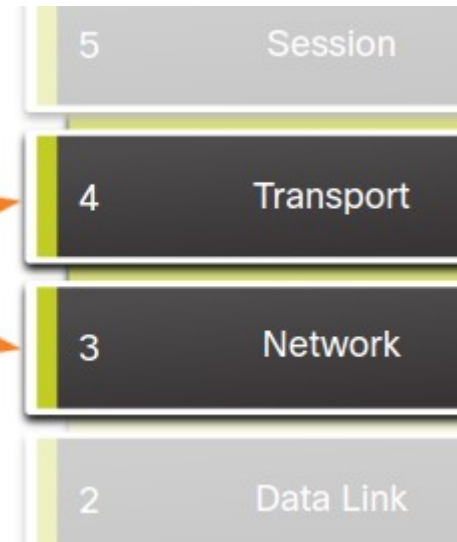
- Definition eines Regelwerks um bestimmte Pakete eines Datenstroms erkennen zu können

- Anwendung einer ACL z. B. für
 - Aufbau einer Paketfilter-Firewall
 - Festlegung welche Hosts bestimmte Netzwerkdienste nutzen dürfen
 - Identifizierung u. Klassifizierung von Paketen für QoS
 - Bestimmung von IP-Adressen für NAT
 - ...

Welche Kriterien stehen ACLs zur Verfügung?

- Klassische **Layer-3**-Daten ...
 - Source- und Destination-Adresse – z.B. IP-Adresse
 - ICMP-Message-Typ (z. B. „echo“ für Ping)
 - Nutzlastprotokoll: IP, IPX, Appletalk...
- ... aber auch **Layer-4**-Daten ...
 - Source- und Destination-Ports (z. B. „*WWW*“ für Port 80)
 - Flags aus dem Layer 4 Header (z. B. SYN, ACK, RST)
- ... oder Wochentage, Uhrzeiten, ...
- ...

Packet filtering works at
Layer 3 and Layer 4



- Standard ACLs können ausschliesslich Layer 3 Pakete filtern indem sie die Quell IPv4 Adressen verwenden.
Es wird also lediglich geschaut „von woher kommen die Pakete?“
- Extended ACLs können aufgrund der Layer 3 Quell- und Ziel IPv4 Adressen filtern.

Zusätzlich ist es möglich auch nach Layer 4 Informationen zu filtern. Wie z.B. TCP-,UDP-Ports und optionale Protokolltyp Informationen. Dadurch kann eine detailliertere Filterung vorgenommen werden.

~~Extended ACLs sind ausserhalb unseres Curriculums!~~

- ACLs werden mit einem **Interface** verknüpft und bei eingehendem oder ausgehendem Datenverkehr angewendet.
- Eingehende ACLs werden verarbeitet **bevor** das Paket geroutet wird (reduziert unnötige Routing-Lookups).
- Ausgehende ACLs werden verarbeitet **nachdem** das Paket geroutet wurde.
- Gesamtzahl der ACLs pro Router: pro Int., pro Richtung und pro Protokoll (Bsp: 2 Interf., zwei Richtungen, zwei Protokolle → 8 ACLs möglich)



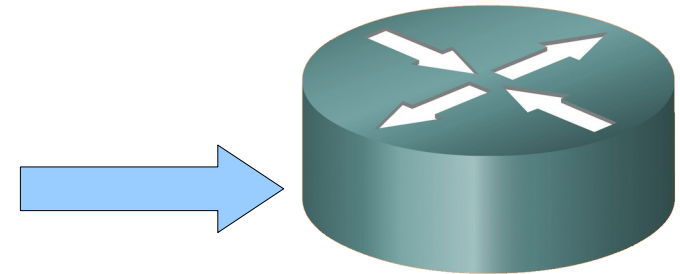
Was genau ist denn eine ACL?

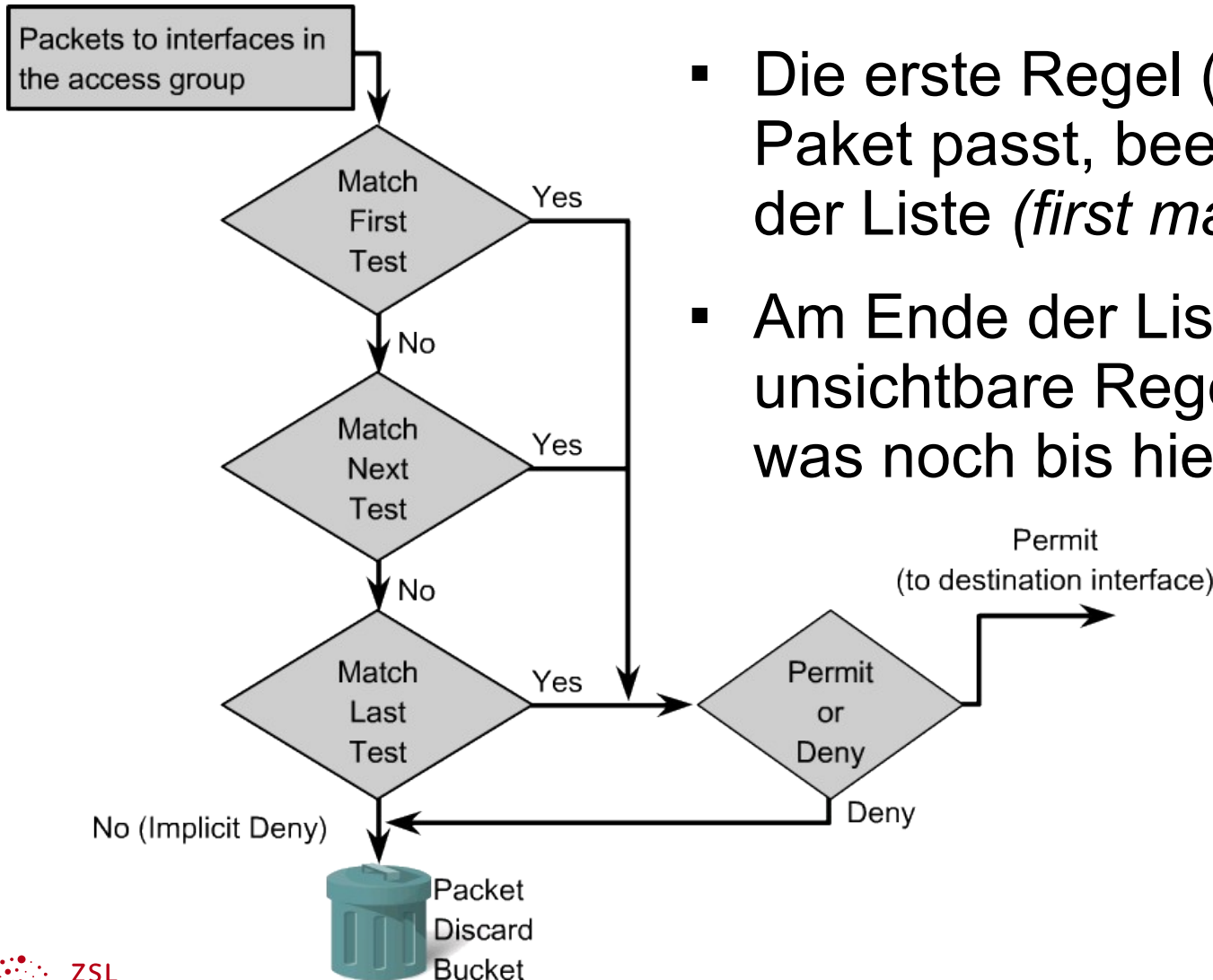
- ACL → geordnete Liste zusammen gehörender Regeln
- Einzelne Regel → Access Control Entry (ACE)
- Beispiel:

```
R1(config)# access-list 2 deny    host 172.16.1.1
R1(config)# access-list 2 permit 172.16.1.0    0.0.0.255
R1(config)# access-list 2 deny    172.16.0.0    0.0.255.255
R1(config)# access-list 2 permit any
```

Die ACL an ein Interface binden:

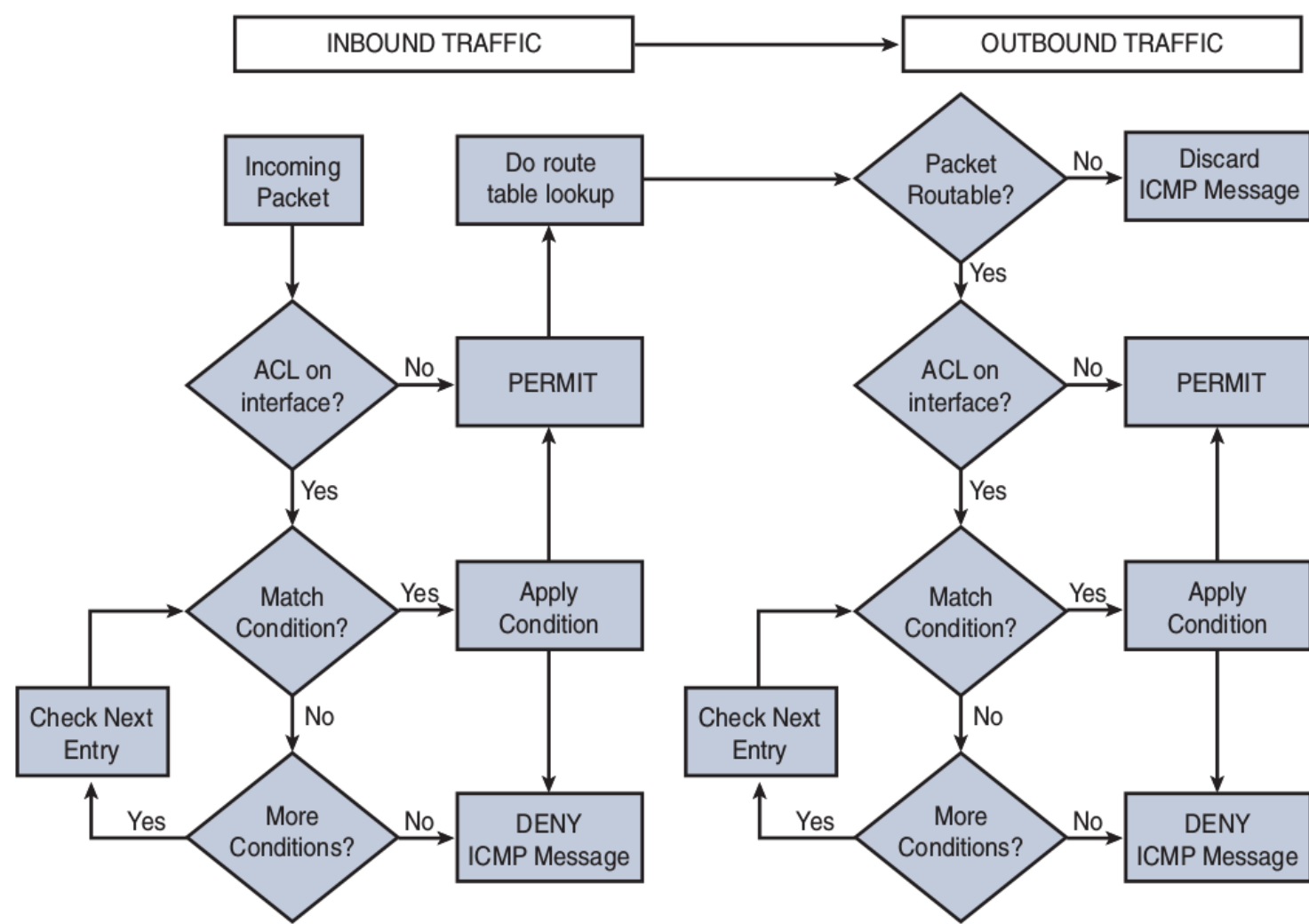
```
R1(config-if)# ip access-group 2 in
```





- Die erste Regel (ACE), die zu einem Paket passt, beendet die Abarbeitung der Liste (*first match logic*).
- Am Ende der Liste gibt es immer eine unsichtbare Regel, die alles „denied“ was noch bis hierher kam!

ACL auf Eingang -> Routing -> ACL auf Ausgang



Quelle: Johnson, A.
(2017) 31 Days Before
Your CCNA Routing &
Switching Exam. Cisco
Press.

```
Router(config)#access-list access-list-nummer  
[deny | permit | remark]  
source [source-wildcard] [log]
```

- **access-list-nummer** – legt Typ der ACL fest. Unterscheidung nach Fähigkeit der ACL (Standard- od. Extended-ACL) bzw. nach Protokoll (IP, IPX, ...)
- **source** – legt geforderte IP-Adresse des **Absenders** fest. Relevante Bits der Adresse werden über Wildcard definiert
- **source-wildcard** – Falls **source**-Bit für Entscheidung relevant = 0, irrelevant = 1
- Bsp.: **access-list 2 permit 172.16.1.0 0.0.0.255**

Was sind Wildcard-Masks (Platzhaltermasken) – Teil 1

Wildcard Mask	Last Octet (in Binary)	Meaning (0 - match, 1 - ignore)
0.0.0.0	00000000	Match all octets.
0.0.0.63	00111111	• Match the first three octets • Match the two left most bits of the last octet • Ignore the last 6 bits
0.0.0.15	00001111	• Match the first three octets • Match the four left most bits of the last octet • Ignore the last 4 bits of the last octet
0.0.0.252	11111100	• Match the first three octets • Ignore the six left most bits of the last octet • Match the last two bits
0.0.0.255	11111111	• Match the first three octet • Ignore the last octet

Wildcard Maskenbit =0
→ der zugehörige Bitwert der Adresse **trifft zu**, muss genau passen!

Wildcard Maskenbit =1
→ der zugehörige Bitwert der Adresse **wird ignoriert**, kann beliebig sein!

Drei Anwendungsbereiche von Wildcard Masks

Nur **eine Adresse** soll exakt ausgewählt werden:

	Decimal	Binary
IPv4 address	192.168.1.1	11000000.10101000.00000001.00000001
Wildcard Mask	0.0.0.0	00000000.00000000.00000000.00000000
Permitted IPv4 Address	192.168.1.1	11000000.10101000.00000001.00000001

Alle Adressen eines bestimmten **Netzwerks** sollen passen:

	Decimal	Binary
IPv4 address	192.168.1.1	11000000.10101000.00000001.00000001
Wildcard Mask	0.0.0.255	00000000.00000000.00000000.11111111
Permitted IPv4 Address	192.168.1.0/24	11000000.10101000.00000001.00000000

Ein ausgesuchter **Adressbereich** soll genau betroffen sein:
(von - bis)

	Decimal	Binary
IPv4 address	192.168.16.0	11000000.10101000.00010000.00000000
Wildcard Mask	0.0.15.255	00000000.00000000.00001111.11111111
Permitted IPv4 Address	192.168.16.0/24 to 192.168.31.0/24	11000000.10101000.00010000.00000000 to 11000000.10101000.00011111.00000000

Berechnung von Wildcard-Masks bei CIDR

Vorgehensweise: Jedes Oktett mit 255 subtrahieren!

Subnetzmaske	Wildcard-Mask
255.255.255.240	0.0.0.15
255.255.254.0	0.0.1.255
/30	0.0.0.3
/25	0.0.0.127

Kontinuierliche Wildcard-Mask je Oktett: 0, 1, 3, 7, 15, 31, 63, 127, 255
(siehe auch das 4. Oktett bei Broadcastadressen!)

Ggf. bei der verwendeten Netzmaske vorher Address-Summarization anwenden, falls mehrere benachbarte Netze betroffen sind.

Wildcard-Mask kann, im Gegensatz zur Subnet-Mask, nach dem ersten 1-Bit auch wieder 0-Bits haben (gemischt).

IP-Adresse: **00001010 00000001 00000010 00000100**
10.1.2.4

Wildcard-Maske: **00000000 00000000 00000001 00000011**
0.0.1.3

Betroffene IP-Adressen:
Netz 10.1.2.4 /30 **00001010 00000001 0000001_ 000001__**
Netz 10.1.3.4 /30

Folgende Host-Adressen werden also gematcht:
10.1.2.4, 10.1.2.5, 10.1.2.6, 10.1.2.7 (Netz: 10.1.2.4/30) &
10.1.3.4, 10.1.3.5, 10.1.3.6, 10.1.3.7 (Netz: 10.1.3.4/30)

Besonderheit: diskontinuierliche Wildcard-Masks Teil 2 (o.o.s.)

Ein weiteres Beispiel:

IP-Adresse: **10101100 00010000 00000001 00010001**
172.16.1.17

Wildcard-Maske: **00000000 00000000 00000000 11101111**
0.0.0.239

Betroffene IP-Adressen: **10101100 00010000 00000001 ____1____**

0 bis 15
und die Werte
128, 64 und 32
ausgehend von
der fixierten
16

D.h. folgende Host-Bereiche im Netz 172.16.1.0/24 sind betroffen:
16-31, 48-63, 80-95, 112-127, 144-159, 176-191,
208-223, 240-254

So können die unterschiedlichsten IP-Adressbereiche herausgegriffen werden!

- **172.16.15.5 0.0.0.0** → Alle Bits müssen passen / eine IP-Adresse / ein Host.

Kurzform → **host** 172.16.15.5

Beispiel:

access-list 2 permit host 172.16.15.5

Schlüsselwort
host darf
auch fehlen

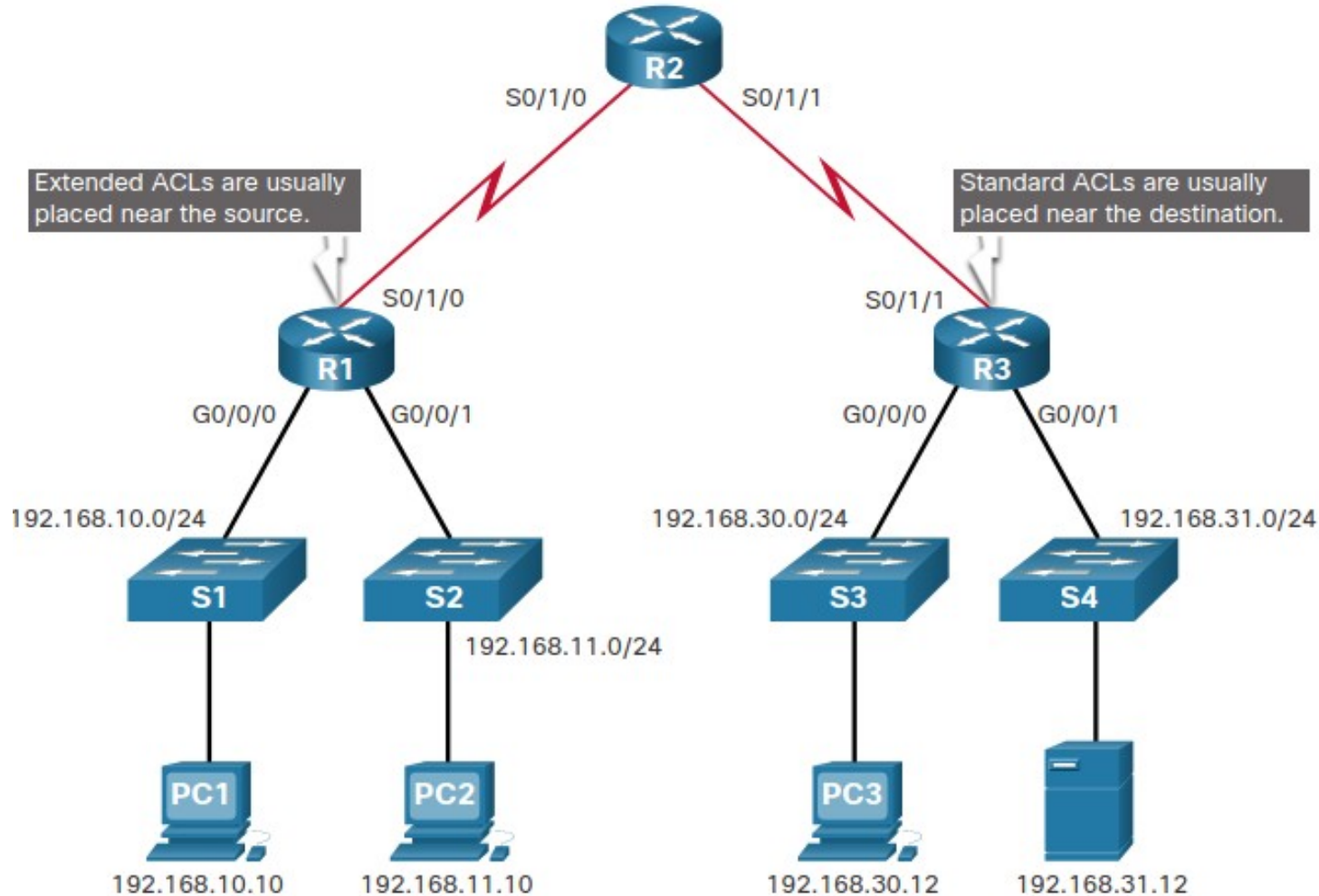
- **172.16.15.5 255.255.255.255** → Alle Bits können ignoriert werden. Es kann auch als **0.0.0.0 255.255.255.255** geschrieben werden. Hierfür gibt es die Kurzform → **any**

Beispiel:

access-list 2 permit any

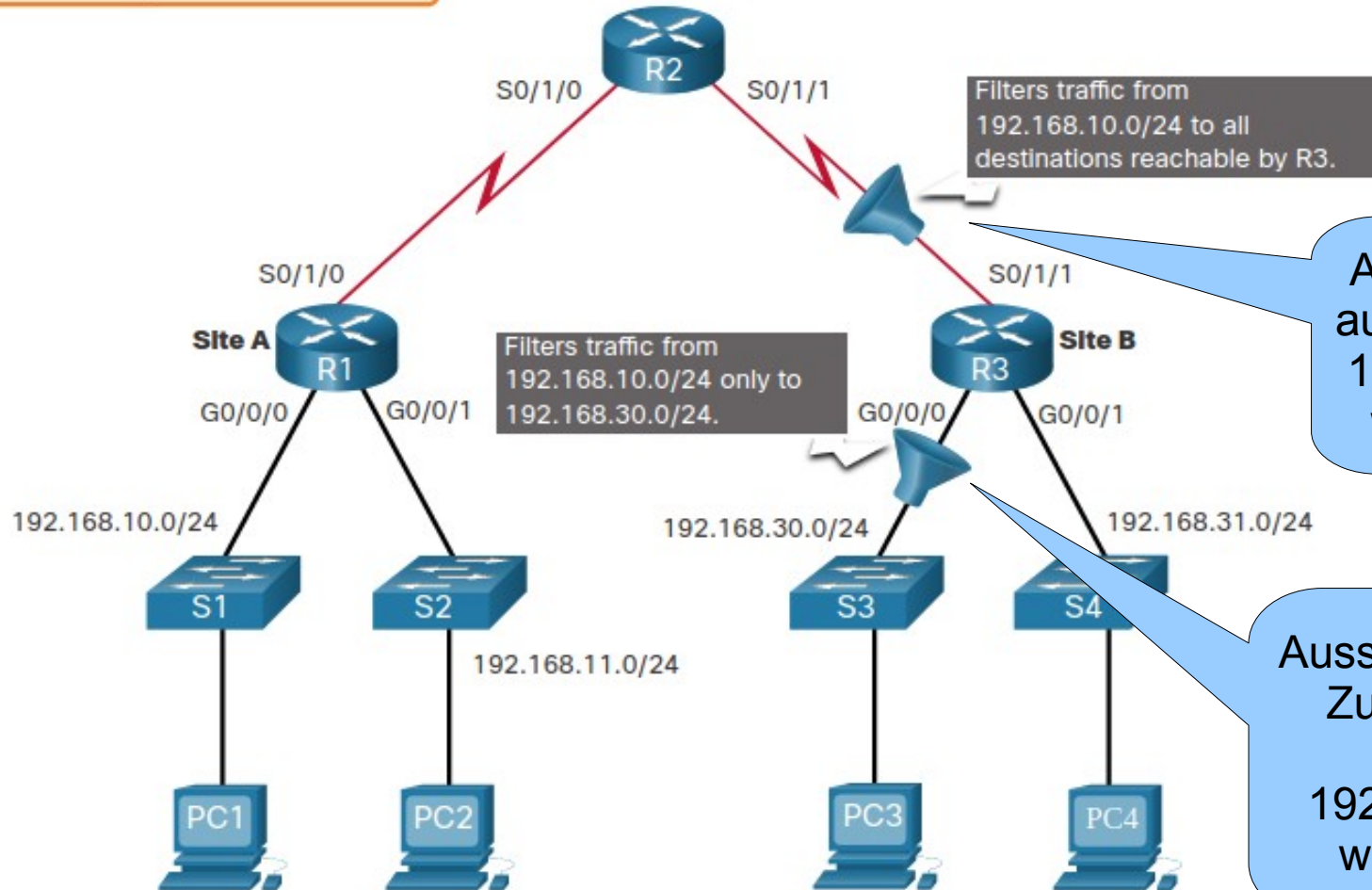
- **Numbered ACLs**
 - 1-99, 1300-1999 → Standard ACLs für IP
 - 100-199, 2000-2699 → Extended ACLs für IP
 - andere Nummernbereiche für andere Protokolle
- **Named ACLs** (ebenfalls „Standard“-und „Extended“-ACLs sowie für verschiedene Protokolle mgl.)
 - **`ip access-list {standard|extended} <name>`**
 - Im nun folgenden NACL-Mode gleicher Syntax wie numbered ACLs – aber Start direkt mit **`permit/deny`**

Wo sollen ACLs plziert werden?



Beispiel für eine Standard ACL am richtigen Ort

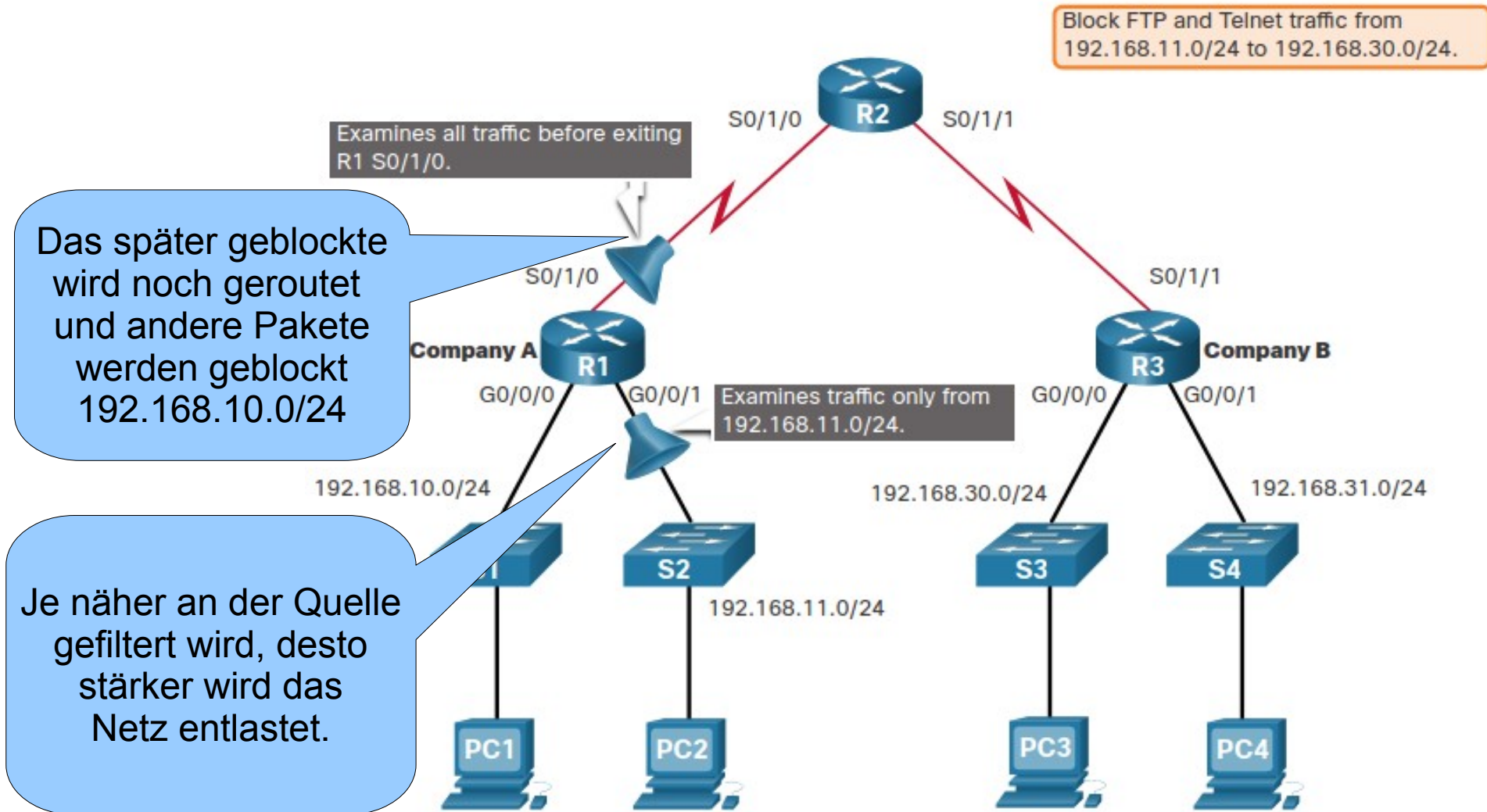
Block all traffic from 192.168.10.0/24 to 192.168.30.0/24.



Auch der Zugriff auf das Netzwerk 192.168.31.0/24 wäre geblockt

Ausschliesslich der Zugriff auf das Netzwerk 192.168.30.0/24 wird geblockt

Beispiel für eine Extended ACL am richtigen Ort



- Alle Modulquizzes
- 4.1.4 PT – ACL-Demonstration

Fragen?

